COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	09/12/2026	**DATE SUBMITTED:**	09/12/2026
GROUP	6	**INSTRUCTOR:**	ENGR. JIMLORD M. QUEJADO

SW: 8.1 USING SORTING ALGORITHMS PT2

1. PROCEDURE OUTPUT

What procedures did you perform?

Objectives: To create a program in C++ using sorting algorithms.

1. To be familiarized with the sorting algorithms.

2. To be able to different the types of sorting algorithms.

3. To be able to create a program with sorting algorithms.

Answer the following questions:

1. Explain the quick sort, shell sort and merge sort types of sorting algorithms.

2. Give simple sample programs in C++ that uses the above sorting algorithms. Use user input of 10 integer values as the array elements to be sorted. Explain how the programs work.

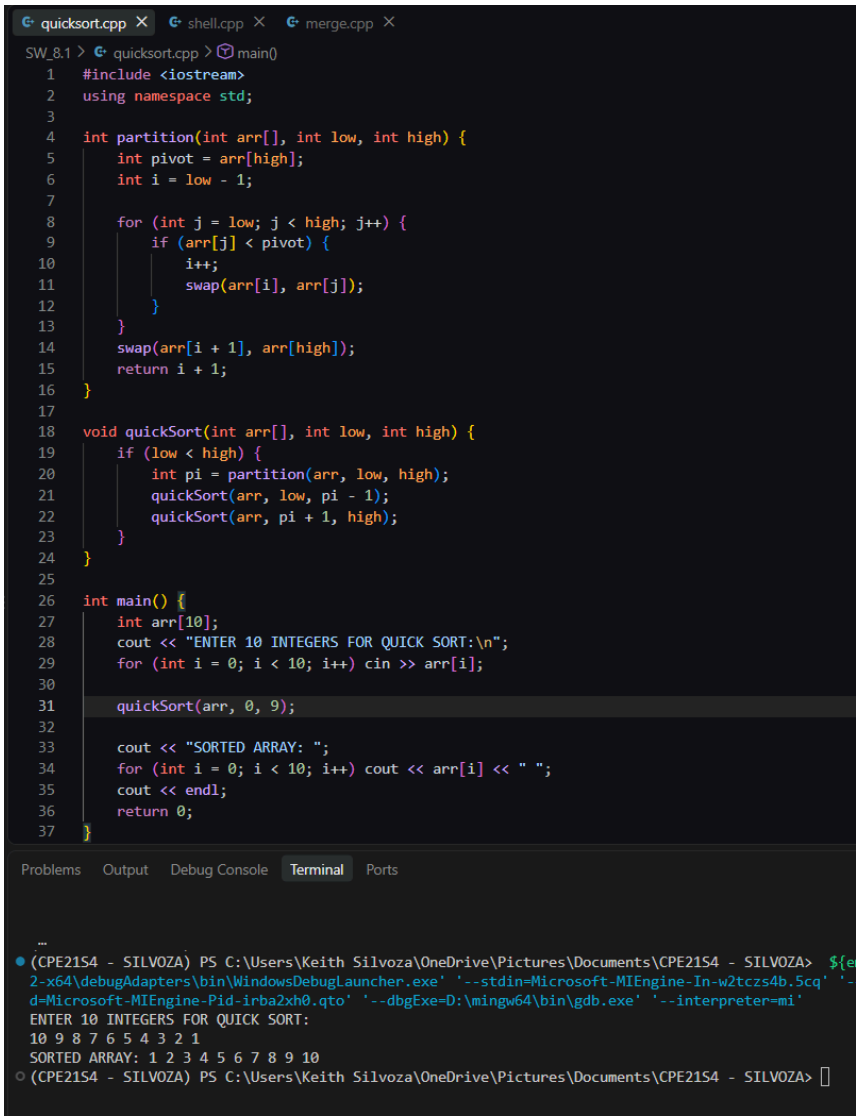
2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ALGORITHM	HOW IT WORKS?	BEST USED WHEN
QUICK SORT	PICKS A PIVOT ELEMENT, PUTS SMALLER ITEMS ON ITS LEFT, LARGER ITEMS ON ITS RIGHT, THEN REPEATS RECURSIVELY ON BOTH HALVES.	SPEED IS PRIORITY; WORKING IN-MEMORY WITH GENERAL DATA.
MERGE SORT	SPLITS THE ARRAY IN HALF RECURSIVELY UNTIL ELEMENTS ARE ISOLATED, THEN MERGES SORTED HALVES BACK TOGETHER.	PREDICTABLE O(n\log n) PERFORMANCE IS NEEDED; SORTING LINKED LISTS OR LARGE EXTERNAL DATA.
SHELL SHORT	COMPARES AND SWAPS	MEMORY IS STRICTLY LIMITED;

	ELEMENTS FAR APART (USING A SHRINKING GAP SEQUENCE) TO MOVE OUT-OF-PLACE ITEMS QUICKLY BEFORE A FINAL INSERTION SORT PASS.	SORTING SMALL TO MEDIUM DATASETS WITHOUT RECURSION OVERHEAD.

CODE
QUICKSORT.CPP



```cpp
#include <iostream>
using namespace std;

int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return i + 1;
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    int arr[10];
    cout << "ENTER 10 INTEGERS FOR QUICK SORT:\n";
    for (int i = 0; i < 10; i++) cin >> arr[i];

    quickSort(arr, 0, 9);

    cout << "SORTED ARRAY: ";
    for (int i = 0; i < 10; i++) cout << arr[i] << " ";
    cout << endl;
    return 0;
}
```

```
(CPE21S4 - SILVOZA) PS C:\Users\Keith Silvoza\OneDrive\Pictures\Documents\CPE21S4 - SILVOZA>  ${e
2-x64\debugAdapters\bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-w2tczs4b.5cq' '-
d=Microsoft-MIEngine-Pid-irba2xh0.qto' '--dbgExe=D:\mingw64\bin\gdb.exe' '--interpreter=mi'
ENTER 10 INTEGERS FOR QUICK SORT:
10 9 8 7 6 5 4 3 2 1
SORTED ARRAY: 1 2 3 4 5 6 7 8 9 10
(CPE21S4 - SILVOZA) PS C:\Users\Keith Silvoza\OneDrive\Pictures\Documents\CPE21S4 - SILVOZA>
```

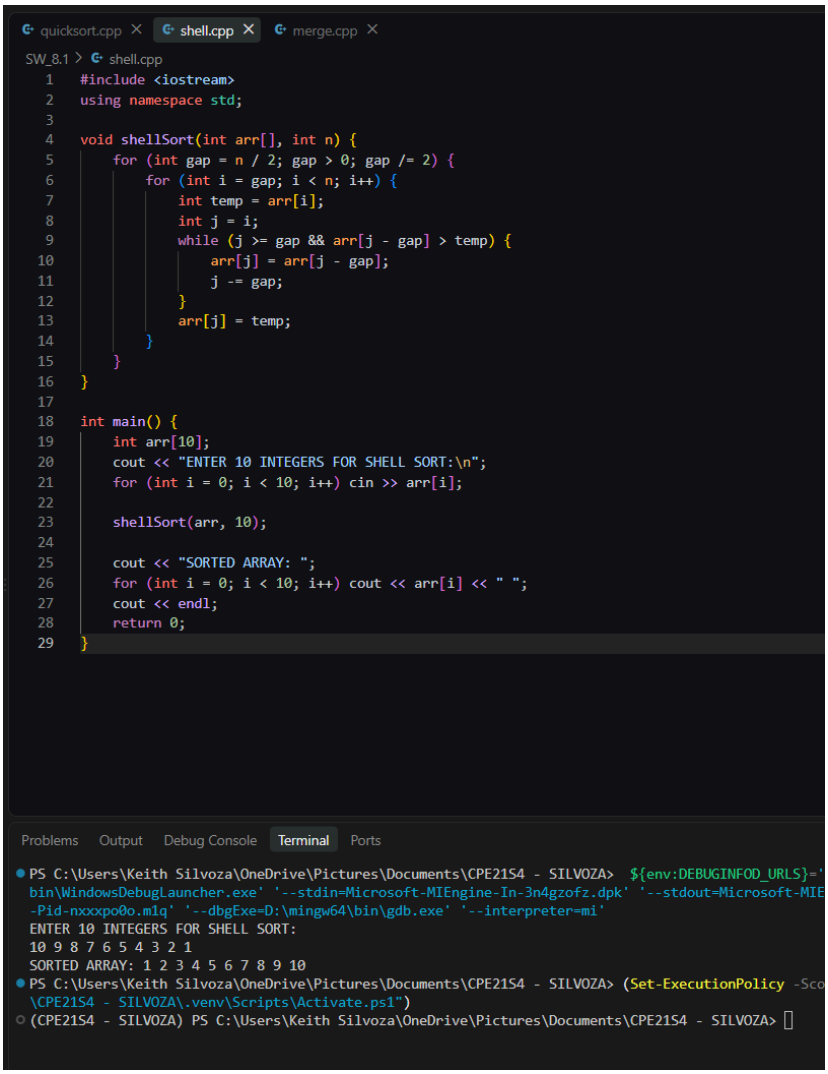
HOW IT WORKS?

- `partition()` **PUTS THE PIVOT IN THE MIDDLE WITH SMALLER NUMBERS ON THE LEFT.** `quickSort()` **KEEPS CALLING** `partition()` **ON THE LEFT AND RIGHT SIDES.**

COMPUTER ENGINEERING

QUEZON CITY

SHELL SORT



```
quicksort.cpp  shell.cpp  merge.cpp
SW_8.1 > shell.cpp
#include <iostream>
using namespace std;

void shellSort(int arr[], int n) {
    for (int gap = n / 2; gap > 0; gap /= 2) {
        for (int i = gap; i < n; i++) {
            int temp = arr[i];
            int j = i;
            while (j >= gap && arr[j - gap] > temp) {
                arr[j] = arr[j - gap];
                j -= gap;
            }
            arr[j] = temp;
        }
    }
}

int main() {
    int arr[10];
    cout << "ENTER 10 INTEGERS FOR SHELL SORT:\n";
    for (int i = 0; i < 10; i++) cin >> arr[i];

    shellSort(arr, 10);

    cout << "SORTED ARRAY: ";
    for (int i = 0; i < 10; i++) cout << arr[i] << " ";
    cout << endl;
    return 0;
}
```

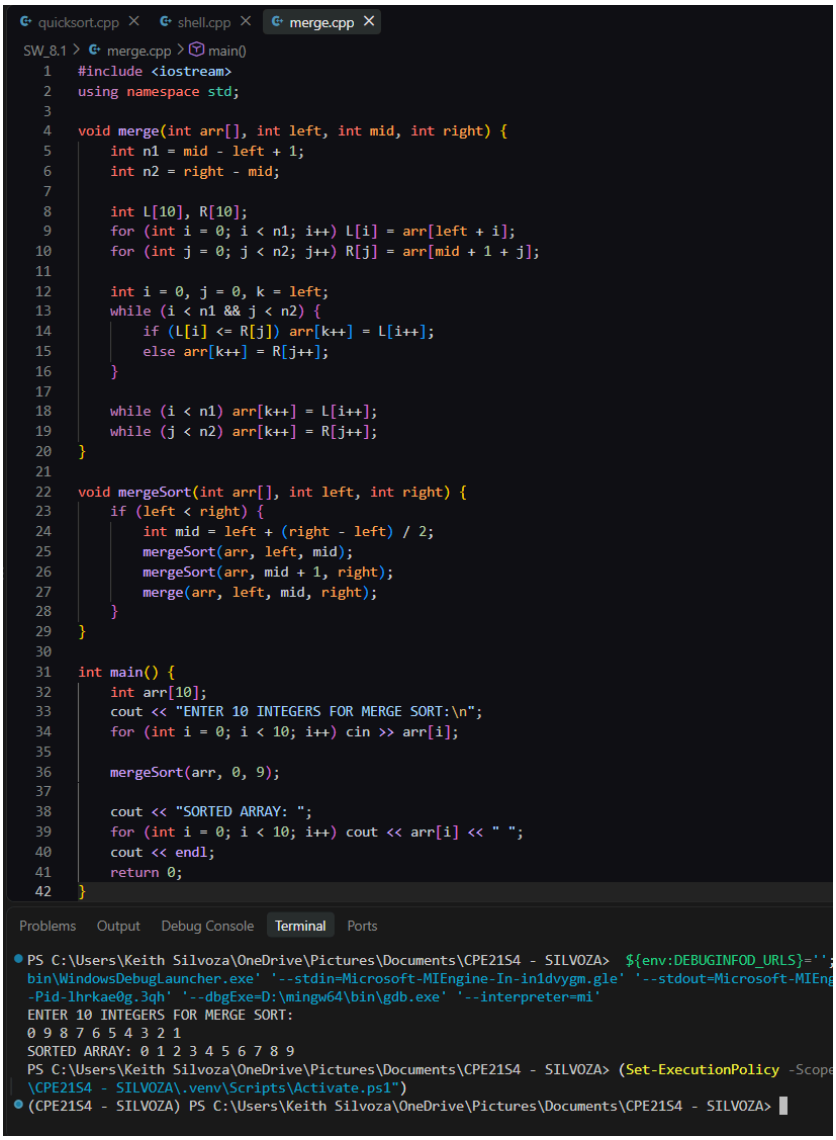
```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\Keith Silvoza\OneDrive\Pictures\Documents\CPE21S4 - SILVOZA>  ${env:DEBUGINFOD_URLS}='
bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-3n4gzofz.dpk' '--stdout=Microsoft-MIE
-Pid-nxxxpo0o.m1q' '--dbgExe=D:\mingw64\bin\gdb.exe' '--interpreter=mi'
ENTER 10 INTEGERS FOR SHELL SORT:
10 9 8 7 6 5 4 3 2 1
SORTED ARRAY: 1 2 3 4 5 6 7 8 9 10
PS C:\Users\Keith Silvoza\OneDrive\Pictures\Documents\CPE21S4 - SILVOZA> (Set-ExecutionPolicy -Sco
\CPE21S4 - SILVOZA\.venv\Scripts\Activate.ps1")
(CPE21S4 - SILVOZA) PS C:\Users\Keith Silvoza\OneDrive\Pictures\Documents\CPE21S4 - SILVOZA>
```

HOW IT WORKS?

- IT STARTS BY SWAPPING NUMBERS 5 SPOTS APART, THEN REDUCES THE GAP TO 2, THEN 1. WHEN GAP IS 1, THE ARRAY IS ALREADY ALMOST SORTED.

MERGE



```cpp
#include <iostream>
using namespace std;

void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    int L[10], R[10];
    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) arr[k++] = L[i++];
        else arr[k++] = R[j++];
    }

    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}

void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

int main() {
    int arr[10];
    cout << "ENTER 10 INTEGERS FOR MERGE SORT:\n";
    for (int i = 0; i < 10; i++) cin >> arr[i];

    mergeSort(arr, 0, 9);

    cout << "SORTED ARRAY: ";
    for (int i = 0; i < 10; i++) cout << arr[i] << " ";
    cout << endl;
    return 0;
}
```

```
PS C:\Users\Keith Silvoza\OneDrive\Pictures\Documents\CPE21S4 - SILVOZA>  ${env:DEBUGINFOD_URLS}='';
bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-in1dvygm.gle' '--stdout=Microsoft-MIEng
-Pid-lhrkae0g.3qh' '--dbgExe=D:\mingw64\bin\gdb.exe' '--interpreter=mi'
ENTER 10 INTEGERS FOR MERGE SORT:
0 9 8 7 6 5 4 3 2 1
SORTED ARRAY: 0 1 2 3 4 5 6 7 8 9
PS C:\Users\Keith Silvoza\OneDrive\Pictures\Documents\CPE21S4 - SILVOZA> (Set-ExecutionPolicy -Scope
\CPE21S4 - SILVOZA\.venv\Scripts\Activate.ps1")
(CPE21S4 - SILVOZA) PS C:\Users\Keith Silvoza\OneDrive\Pictures\Documents\CPE21S4 - SILVOZA>
```

HOW IT WORKS?

- `mergeSort()` SPLITS THE ARRAY IN HALVES. `merge()` COMPARES THE LEFT AND RIGHT HALVES AND COMBINES THEM BACK IN ORDER.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

COMPUTER ENGINEERING

QUEZON CITY

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.